WebQuake

A web application which uses live earthquake reports and live tsunami information to mitigate the short-term effects of an earthquake in Japan, by creating a user-friendly interface to view this data with notifications on an event occurring. It can also provide past earthquake information for informational purposes.

Table of Contents

1 Analysis of the Problem 2
1.1 Problem Identification 2
1.1.1 Describe the features that make the problem solvable by computational methods. 2
1.1.2 Explain why the program is amenable to a computational approach. 2
1.2 Stakeholders 3
1.2.1 Identify and describe those who will have an interest in the solution. 3
1.3 Research the problem 6
1.3.1 Research the problem and solutions to similar problems 6
1.3.2 Describe the essential features of a computational solution. 12
1.3.3 Explain the limitations of the proposed solution. 12
1.4 Specify the proposed solution 13
1.4.1 Specify and justify the solution requirements. 13
1.4.2 Identify and justify measurable success criteria. 14
2 Design 15
2.1 Decompose the problem 15
2.1.1 Break down the problem justifying any decisions made 15
2.1.2 Tree Diagram 16
2.2 Describe the solution 17
2.2.1 Explain and justify the structure of the solution 17
To create an online application, there must be two parts to the development: 17
2.2.2 Describe the parts of the solution using algorithms 19
2.2.3 Identify key variables/data structures/classes 22
2.2.4 File descriptions permanent storage 23
3 Developing the Solution 25
3.1 Iterative Development Process 25
3.1.1 Input Forms 25
3.1.2 Writing to files 27
3.1.3 Processing data 27
3.1.4 Output 28
4 Testing to Inform Development 30
5 Testing to inform evaluation 32
5.1 Testing to inform function and robustness 32
5.1.1 Calculation Test Plan 32
5.1.2 Validation Test Plan 33
6 Evaluation 37
6.1 Success of the solution 37
6.2 Describe the final product 39
6.3 Maintenance and development 41
6.3.1 The maintainability of the solution 41
6.3.2 Further development of the solution 41
Glossary: 42
References: 43

1 Analysis of the Problem

1.1 Problem Identification

People of Japan who primarily use a desktop as their way of communicating with the Internet have no way to get earthquake early warnings – they must be near a smartphone that has signal or a TV tuned to a channel that broadcasts Earthquake Early Warnings, like NHK [1]. In 2023, 70% of people in Japan owned a smartphone [2]. As TVs must be on and tuned to a specific channel to receive the warning, this leaves around 30% of people not being prepared for strong shaking when it occurs which prevents people from defending themselves from the dangers of an earthquake.

Most Japanese workers work 8 hours a day [3] meaning that for a third of their day, they may have limited access to their phones or a TV, but they have unfettered access to a computer with internet access and therefore access to websites.

Additionally, tourists in Japan who do not speak Japanese must install software (like NERV, or JQuake) to view real-time earthquake & tsunami information in English. Installing software takes time, something you do not want when you need information in a hurry.

My project named WebQuake aims to offer all this information in one easy-to-use web interface. With all text in both English and Japanese, accessing real-time earthquake & tsunami information has never been easier to those in need. No install or account is required so information is quickly accessible, even to those who have not accessed it before, so only a computer with internet access is required.

1.1.1 Describe the features that make the problem solvable by computational methods.

Methods of mitigating the short-term impacts of an earthquake are possible using non-computational methods, like knowing to hide under a table when strong shaking occurs or getting to high ground if you live near the coast and you feel an earthquake. However, these are not very effective as they rely on humans (who make mistakes), so computational methods, like WebQuake, are required to protect as many people as possible. WebQuake will make use of abstraction, by consolidating lots of non-readable information into one easily readable GUI and discarding information which is not immediately useful to the user.

1.1.2 Explain why the program is amenable to a computational approach.

It will also make use of websockets, as an efficient way to continuously transfer data from WebQuake's server to its users.

In the 2011 Tōhoku Earthquake, an Earthquake Early Warning was released approximately 8.6 seconds after the earthquake occurred, giving the people of Sendai 15-20 seconds lead time before very strong shaking happened [4] and they could employ those non-computational methods. Without the EEW, which was created with computational methods, things like hiding under the table would be much less effective, as there would be no 15-20 seconds to prepare, and many more people would have died.

1.2 Stakeholders

1.2.1 Identify and describe those who will have an interest in the solution.

My primary stakeholders are the Japanese public as each and every one of them could be affected by an earthquake. In the modern world, almost everyone has access to a computer with a web browser, with ⅔ of those who use a computer in Japan using them almost every day [5]. Therefore, WebQuake is appropriate for their needs as they will be able to access it quickly. WebQuake also caters to both those who are looking for basic information (like current earthquake/tsunami warnings) and more advanced users who may want information like tsunami observations.

When I interviewed a stakeholder, the following key features were recognised as being the most necessary for users:

- An easily readable interface.
- Both English and Japanese language support.
- Notifications on issuance of emergency warnings (earthquake & tsunami).
- Being able to view past earthquake reports.

Additionally, I asked five residents of Japan online to answer my questionnaire below:

	Have you received an Earthquake Early Warning recently?	Were you at/near a computer with internet access at the time?	Would it be useful to receive a warning on your computer?	Are you able to install software on your work/school computer?	Would you prefer opening a website or installing software to view earthquake information?	Why?	Would you enable notifications for earthquake reports?	Would you enable notifications for an I_{JMA} observation of $\geq$ 5- & tsunamis?	Why (for both notifications)?
Person 1	Yes	Yes	No	No	Opening a website	No installation needed as I cannot install software and its easier.	No	Yes	If I care about an earthquake report, I can open the software. However, it would be useful to be notified of strong shaking - a warning may not come before the shaking.
Person 2	Yes	No	Yes	No	Opening a website	I could open the same familiar interface on both my laptop and phone if I wanted.	Yes	Yes	My family is spread across Japan so I would like to know if an earthquake has affected them.
Person 3	Yes	No	Yes	No	Opening a website	My school does not let me install apps.	Yes	Yes	I live in a very earthquake prone region so any notifications which can prepare me are useful.
Person 4	Yes	Yes	Yes	Yes	Installing software	I can have the software easily open automatically when I turn my computer on.	Yes	Yes	I have a large interest in my countries earthquake system, it fascinates me.
Person 5	No	N/A	Yes	Yes	Opening a website	I already have a web browser installed, and I don't like installing new things.	No	Yes	Report notifications are not useful to me - I am rarely affected by earthquakes. But if I am affected, I'd like to know as soon as possible.

Four out of five people I asked had recently received an Earthquake Early Warning and of those people, half were near a computer at the time. This means that, *out of my limited test group*, half would have been able to access WebQuake at the time of the earthquake.

Eighty percent of my respondents said that it would be useful for them to receive a warning on their computer – this is one of the key features of WebQuake and it is nice to know that it will be well used.

Three out of five people are not able to install software on their work/school laptop – this means that only WebQuake and Scratch R.E.V. will be accessible to them. Scratch R.E.V. is only available in Japanese, so WebQuake will be useful to anyone who only speaks English (and those who speak Japanese as WebQuake is available in Japanese).

The majority of my respondents said that they'd prefer to open a website to get earthquake information as opposed to installing software and provided reasons such as: it's easier not to install software, I don't like installing new things and that I could look at the same familiar interface on a computer and my phone. People often dislike installing software to try a service, in fact 68.5% of people are 'frustrated' or more when a business requires you to download an app. [14] Everyone who uses Windows, MacOS or ChromeOS (90% of desktop users in February 2024 [15]) has a web browser pre-installed with their operating system, so no installation of any kind is required to access WebQuake.
Furthermore, it has been stated that "familiarity results in a truncated neural response" [16], or in this context, when users are familiar with the software, they can navigate it and get to what they need to quicker. This works in favour of WebQuake for 2 reasons: people can use the web browser they are familiar with to access WebQuake quicker in times of disaster and people can use WebQuake on both their computers and mobiles – they don't have to use different and unfamiliar interfaces based on what device they happen to be using at the time.

Many of the people I asked said that they would enable notifications for earthquake reports, one of which saying that their family lives all over Japan so, with notifications, they could see if their loved ones were affected.

All the above feedback will be used during development to make WebQuake more widely appeal to my demographic.

1.3 Research the problem

1.3.1 Research the problem and solutions to similar problems

There are four programs which are similar to WebQuake: GlobalQuake, JQuake, Scratch Realtime Earthquake Viewer and NERV.

GlobalQuake provides unofficial earthquake predictions using seismic points from around the world. This program can be computationally intensive to run as, unless you connect to the server, all prediction calculations are done locally – this requires a decent amount of processing power and >3GB of free RAM (if you want to load all seismic points from around the globe). Moreover, GlobalQuake is not specific to Japan which means that it only has access to a limited number of seismic points in Japan, meaning that predictions that are made, while usually accurate, come significantly later than official ones. GlobalQuake relies on (mostly) community-driven & unofficial seedlinks (services set up by the public to send data from their seismometers). These seedlinks can be unreliable and could be taken down at any point by their owners, which has the potential to create 'blackspots' where people cannot see what is happening in a region as the seedlink for it is not responding. GlobalQuake is not accessible on a mobile device.

JQuake provides official earthquake reports along with official seismic points (but no numerical intensity labels, instead it is shown as a colour), and tsunami warnings displayed on a map. It also lists past earthquakes and their magnitude, location, and depth. However, it is not web-based, it is software which requires installation, meaning it cannot be accessed (for the first time) in a hurry – something which people often need as few people will pre-emptively install a program of this nature. Additionally, JQuake is not accessible on a mobile device, meaning that anyone not actively using a computer will not have access to it.

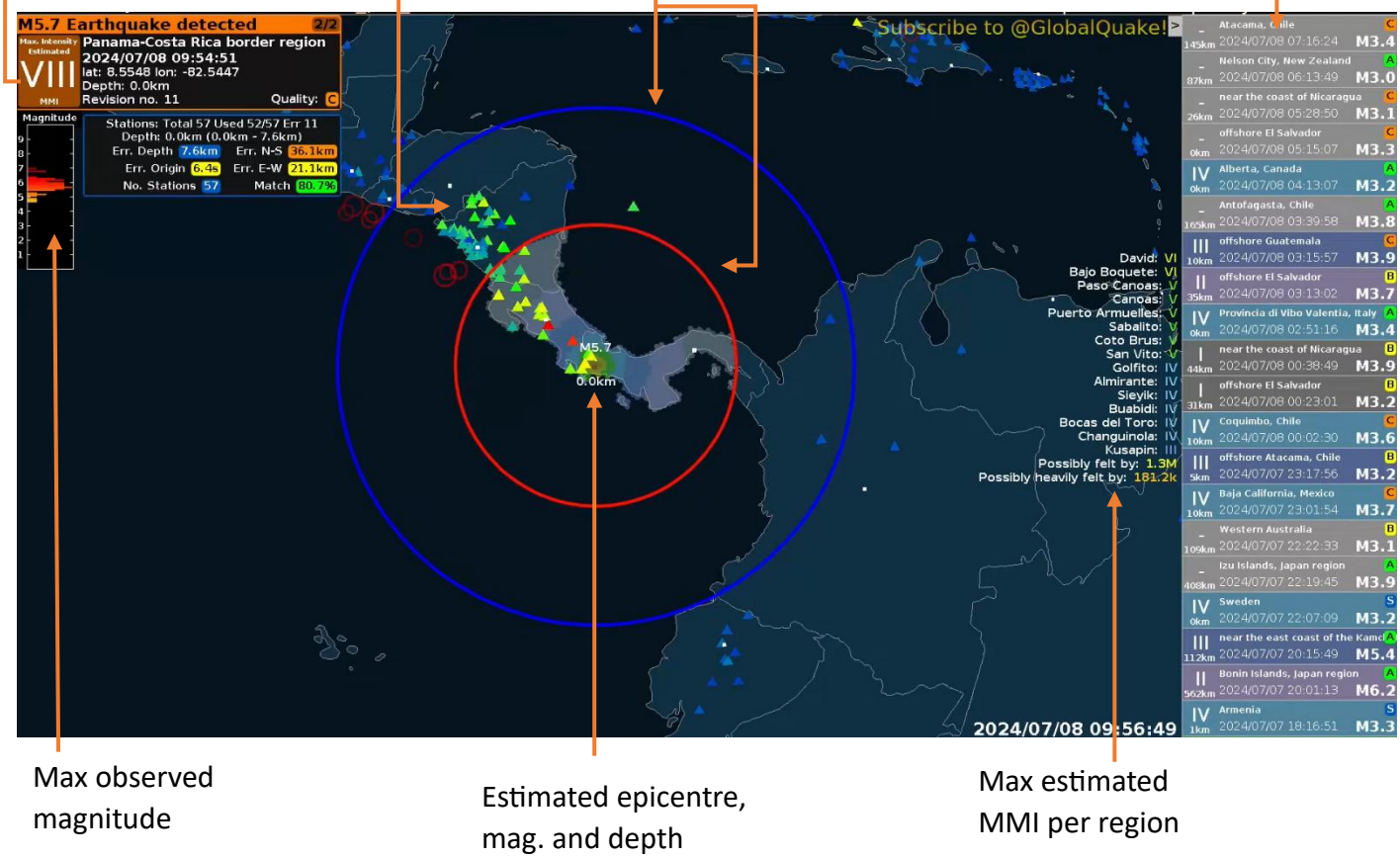
Scratch Realtime Earthquake Viewer is web-based (as it was developed using Scratch – a web-based programming interface) and it provides official seismic points with numerical intensity labels, tsunami warnings with observations, long-period ground motion and interim earthquake reports among other useful information. However, as it is coded in Scratch, a program of this size often takes >30 seconds to load in ideal conditions (and ideal conditions are rare after a large earthquake – networks will be congested, etc…), and fast access to potentially live saving information is particularly important. Furthermore, when I tested Scratch R.E.V. on my iPhone, it often crashed when interacting with any information. Because of this, it may not be ideal for use in a disaster, when getting the information you require with minimal disturbance is key.

NERV is a mobile app that provides any disaster information to do with Japan, from weather warnings to missile alerts. Earthquake & tsunami information is included in this, with it providing official earthquake reports, seismic points (with no numerical label, rather a key which maps the point's colours to an intensity value), tsunami warnings and observations. This is also the only program on the list that can deliver notifications, however it is slightly limited. Notifications can only be provided for up to three municipalities (for free) & your current location in Japan. A positive of NERV is that, as it is a mobile app, "critical" notifications can be delivered, something not possible in WebQuake due to the limitations of a web-app. Critical notifications play a sound at full volume to alert the user of danger, regardless of the user's notification settings.

Detailed Comparisons

GlobalQuake [9]:

Positives	Negatives
• Shows detected earthquakes. • Shows the estimated epicentre, magnitude, intensity, and depth of earthquakes. • Works worldwide	• Predictions are made by the software, not an official source, so can be inaccurate. • Requires a lot of RAM and computational power as predictions are made locally. • Seismic points are community run so can take a long time to update which leads to slow predictions. • Not specific to Japan so much less accurate for earthquakes in Japan. • Not web-based. • Cannot provide notifications.



JQuake [10]:

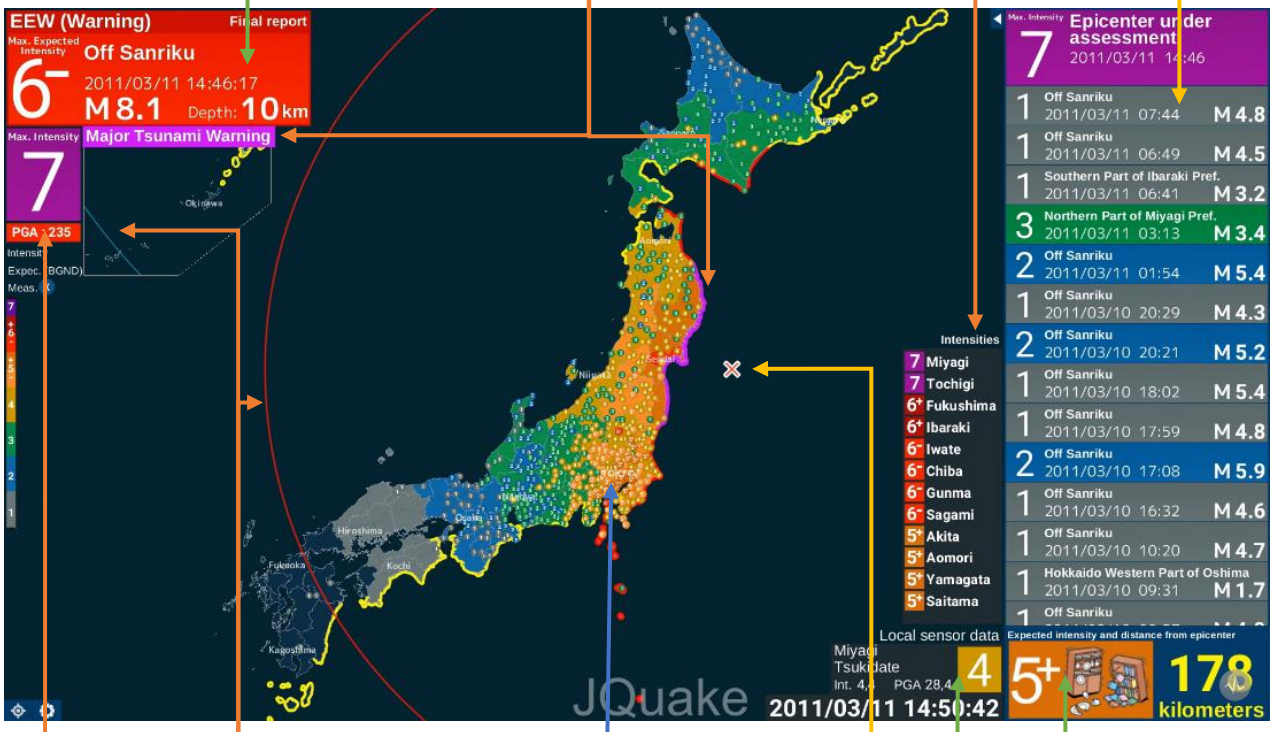
Positives	Negatives
• Shows official earthquake reports made by the Japan Meteorological Agency. • Shows the official estimated epicentre, magnitude, intensity & depth. • Shows NIED seismic points. • Shows tsunami warnings. • Both English & Japanese languages	• Not web-based. • Does not translate the seismic points into an intensity. • Does not show official estimated long period ground motion. • Does not show tsunami observations. • Does not show the current tsunami status of an earthquake (estimation underway or warning given). • Does not show interim seismic reports. • Cannot provide notifications.

Earthquake Early Warning viewer with estimated I_{JMA}, location, magnitude, depth, report no. and date & time

Tsunami Warning (map & text)

Past earthquake list

Max. observed I_{JMA}



Max. observed I_{JMA} & PGA.

P & S-wave visualisation

Map interface with seismic points.

Est. epicentre of current EEW

Estimated & observed I_{JMA} at a place of your choice

Scratch Realtime Earthquake Viewer [11]:

Positives	Negatives
• Shows official earthquake reports made by the Japan Meteorological Agency. • Shows the official estimated epicentre, magnitude, intensity & depth. • Shows NIED seismic points with intensity. • Web-based. • Shows official estimated long period ground motion. • Show tsunami observations. • Shows the current tsunami status of an earthquake. • Shows interim seismic reports.	• Is coded in Scratch so takes >30 seconds to load – not useful when you need quick earthquake info. • Only in Japanese – it also cannot be easily translated from Japanese as any text is drawn in using SVGs. • Cannot provide notifications. • Has been known to go down during periods of high seismic activity as too many users try to access it.

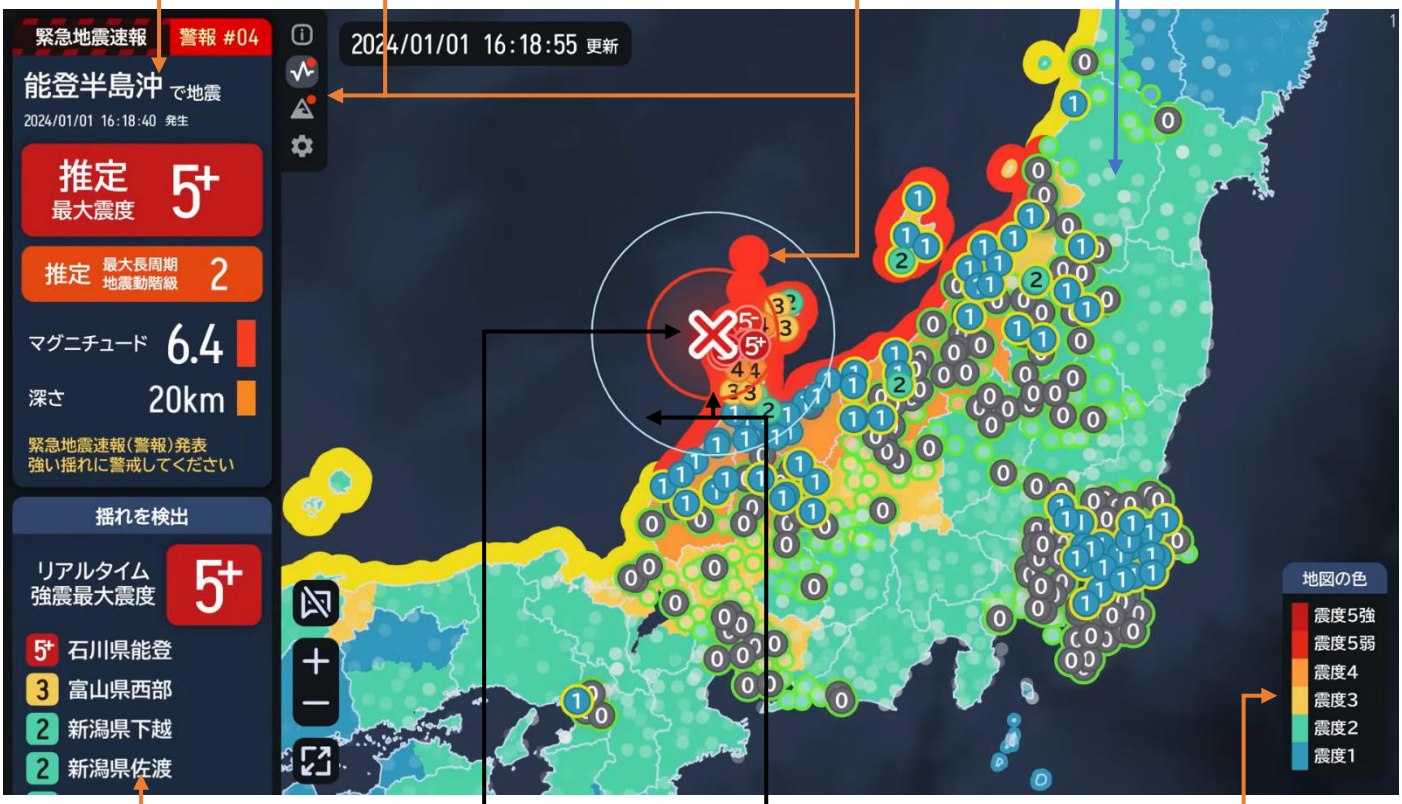
Earthquake Early Warning viewer with estimated I_{JMA}, location, magnitude, LPGM, depth, report no. and date & time.

Tab interface:

- Past earthquakes
- Current earthquakes
- Tsunami information
- Settings

Tsunami warning (map and dot on tsunami tab)

Map interface with seismic points



Max. observed I_{JMA}

Est. epicentre of current EEW

P & S-wave visualisation of current EEW

Colour to I_{JMA} key

NERV [12]:

Positives	Negatives
• Shows official earthquake reports made by the Japan Meteorological Agency. • Shows the official estimated epicentre, magnitude, intensity & depth. • Shows NIED seismic points. • Both English & Japanese languages. • Notification support.	• Mobile only. • Only notifications that apply to your location in Japan or three chosen prefectures are sent. • Does not translate the seismic points into an intensity. • Does not show official estimated long period ground motion. • Does not show tsunami and earthquake information in the same interface. • Paid option available.



Feature Comparison Table

App	Feature													
	Specific to Japan	Shows official reports	Shows mag., location, depth	Shows NIED points	Shows point intensity	Web based	Shows est. LPGM	Shows tsunami info	Shows tsunami obs.	English option	Notifi-cations	No paywall	Official info	Light-weight program
WebQuake	✓	✓	✓			✓	✓	✓	✓	✓	✓	✓	✓	✓
GlobalQuake			✓							✓		✓		
JQuake	✓	✓	✓	✓				✓		✓		✓	✓	✓
Scratch R.E.V.	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓	✓	✓
NERV	✓	✓	✓	✓				✓	✓	✓	½*		✓	✓

*Notifications can only be sent for your current location (Japan only) or for up to 3 chosen municipalities (or up to 6 with a subscription [13]).

1.3.2 Describe the essential features of a computational solution.

Some features of the solutions I have researched I think will fit well into WebQuake. These include:

- Web based design – my users will not need to install anything to access WebQuake. This means that there is no lengthy install process to go through when they want to access the information that WebQuake offers.
- User friendly interface – if users must spend time navigating a clunky interface, they will have less time to interpret the information WebQuake is giving them to prepare themself.
- Localisation support – Everything must be translated into Japanese as well as English. If the user cannot understand English then without a Japanese translation, WebQuake would be useless to them. The user will have the option to select either language and switching between them will be seamless.
- Notifications – WebQuake will send web notifications to those who opt-in to them. It is not a realistic expectation for people to have WebQuake open all the time, so a notification upon an event occurring will prompt them to open the webapp to see more information.

1.3.3 Explain the limitations of the proposed solution.

If an earthquake happens at a shallow depth (<50km) and the epicentre is directly below land, for the people at/near the epicentre, the shaking will often start before the warning is issued. This is a limitation of the Japanese EEW system and cannot be resolved (for now).

It is possible that the JMA will falsely issue an EEW. WebQuake will send a notification whenever an EEW is issued as it is assumed that everything issued by the JMA is accurate. Since 2011, only five erroneous EEWs have been released out of hundreds [6], so false EEWs are very unlikely – especially with recent improvements.

WebQuake will only be able to provide its information for Japan as information of this kind for other countries (like Taiwan and the West Coast of America) is not as readily available and detailed.

WebQuake will require a constant, stable internet connection to access. While Japan's internet penetration rate stood at 94% in 2022 [8], in times of extreme disaster, services like internet can fail or be overloaded, rendering WebQuake inaccessible to them. Additionally, WebQuake will warn users if they have an unstable internet connection, and prompt them to refresh the page if their connection to WebQuake's server is lost.

1.4 Specify the proposed solution.

1.4.1 Specify and justify the solution requirements.

Hardware requirements: Laptop/desktop/mobile with a web browser. Mouse or touch needed for interaction.

Software requirements: Any operating system with a web browser released after 2012 (minimum compatibility for WebSockets and desktop web notifications). If a user wishes to receive notifications on their mobile device, they must have an iPhone with iOS 16 or later or an Android based phone with a web browser released after 2015.

The live earthquake reports and tsunami information will be taken from a websocket (Project DM-DSS) and turned into values which are suitable to be sent to the frontend via a websocket to interpret. I have chosen websockets over HTTP requests as they are a more efficient way of continuously transferring data to my users.

A user friendly interface is also required to display all information in an easy to understand manner, and dynamically focus the interface on the most important available information while being able to handle multiple pieces of information of the same importance at the same time.

Notifications for an Earthquake Early Warning and Tsunami Warnings/Observations are required to ensure that emergency information will reach as many people as possible during times of disaster. It would not be reasonable to assume that people will always have WebQuake open on their device, so notifications provide a way of warning users of danger without monitoring WebQuake's interface 24/7.

WebQuake will also inform people when an earthquake has a high probability of causing a tsunami. Earthquakes are most likely to cause extreme tidal waves when their epicentres are located beneath the ocean, their magnitude is high (above 6.5) and their depth is low (below 70km) [7]. This is essential to advise and prepare those who live in coastal areas that they should make their way to high ground as it can sometimes be too late to wait for tsunami information to be released.

1.4.2 Identify and justify measurable success criteria.

1. Creates an interface:
    a. Current date & time (JST) – can be used to check your connection to the server.
    b. Shows real time seismic intensity values – for easy-to-understand information.
        i. Interprets the NIED gif into I_{JMA} – values are easier to interpret than colours.
    c. Dynamically focuses on the most important current information – so that the user always receives the information they require.
    d. Shows EEWs – so that users are informed of any emergency warnings:
        i. Shows the estimated location of their epi/hypocentres – so they can see if they are near it.
        ii. Shows their estimated magnitude and depth – so they can see the scale of the earthquake.
        iii. Shows their estimated I_{JMA} per prefecture – so they can prepare for a specific amount of shaking.
        iv. Accurately shows their P/S-waves – a visual representation of when to expect shaking and the strength of shaking associated with each wave.
        v. Shows the report number – so they can see how recent the earthquake was.
        vi. Shows the estimated long-period ground motion – so they can prepare for shaking over an extended period of time, especially if they are in a high-rise building.
        vii. Uses an algorithm to determine whether a tsunami is possible and informs the user of its decision – so users near the coast can act before official warnings are issued.
    e. Shows tsunami warnings – so that affected users can take live-saving action:
        i. Shows the type of tsunami warning – a simple way of seeing the scale of the tsunami.
        ii. Shows the estimated wave heights per prefecture – more detail on the scale of the tsunami.
        iii. Shows the estimated arrival times per prefecture – so they know when to expect it.
        iv. Shows the tsunami observations – so they can see what is happening in reality.
    f. Shows official earthquake reports – so they can see the official information on what happened:
        i. The epicentre location – so they can see where it happened.
        ii. The magnitude and depth – so they can see the scale of what happened.
        iii. The time and date – so they can see when it happened.
        iv. The long-period ground motion – to see if they should expect shaking over a long period of time.
2. Gives notifications:
    a. EEW – so that users are quickly informed of expected strong shaking.
    b. Observation of I_{JMA} of >= 5- – so that users are informed of shaking if an EEW isn't released.
    c. Tsunami warning/advisory – so that users are quickly informed of expected high tidal waves.

2 Design

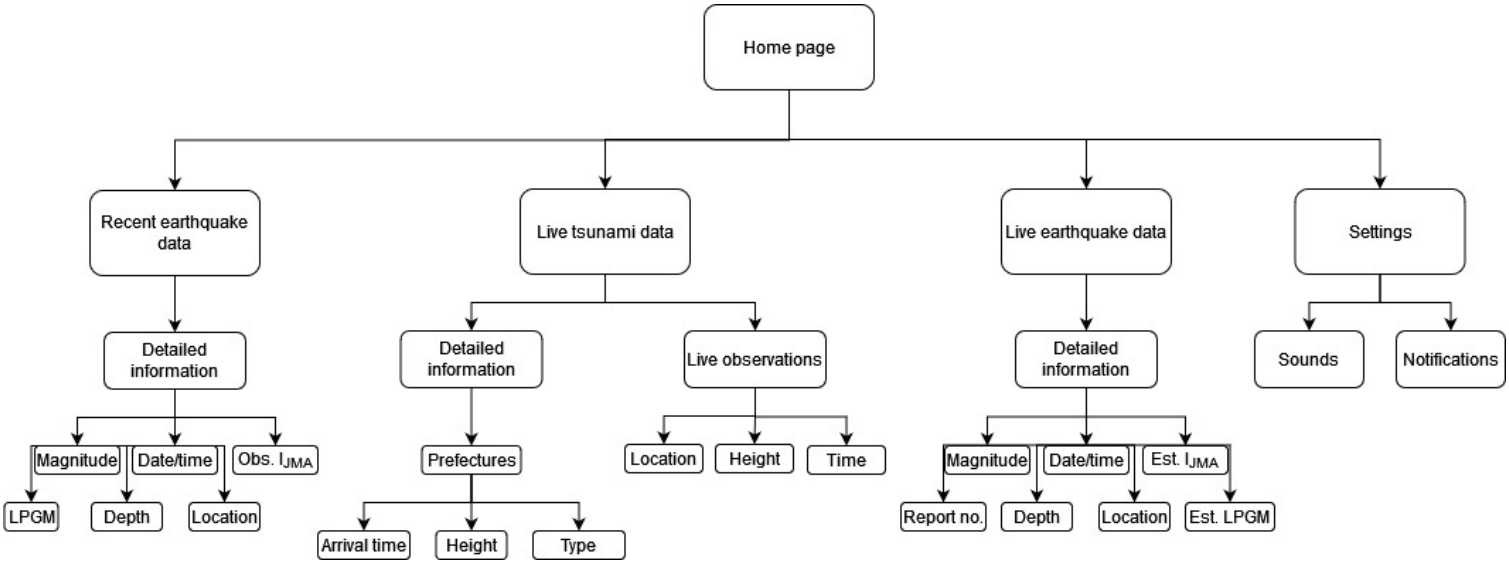
2.1 Decompose the problem

2.1.1 Break down the problem justifying any decisions made

Creating an application capable of effectively mitigating the short-term impacts of an earthquake is not a simple task, and by breaking down the problem, it makes the process of development easier and helps users understand exactly what WebQuake does.

- Communication between the user and WebQuake's server:
    - This is essential as WebQuake must send the data to the user, having the user get the data themselves could incur a cost.
    - The best way to achieve this is through the client-server structure to be used for communication.
    - Real-time communication is crucial for WebQuake as earthquake predictions cannot be made before the earthquake occurs. Every millisecond counts towards more people being prepared.
        - Considering this, a protocol is needed that supports web browsers and is low latency. The best choice would be WebSockets as this ensures that messages received are in order. Earthquake predictions can change quickly, especially soon after occurrence, so messages received out of order could confuse the user.
- Creating an interface to view the data:
    - An easy-to-understand interface is imperative for times of disaster, you cannot waste time figuring out a complicated interface when time is fundamental to preparing yourself.
    - It must balance being information-rich and not overwhelming the user with irrelevant material.
    - The most appropriate way to solve this problem is to only automatically display the most important information to the user at one time.
    - Choosing what is most important to the user is not an easy task, as it can be subjective, so I have chosen an order in terms of how life-threatening the hazard is:
        - Tsunami warnings:
            - Tsunami warnings will always display over anything else when they are made. However, if an earthquake warning is issued during a tsunami warning, the earthquake warning will be displayed only while it is active.
        - Earthquake Early Warnings/forecasts:
            - In the event of multiple EEWs/forecasts, the most recent warning report will be displayed.
        - Earthquake reports:
            - Will be shown for up to 180 seconds after information is received.
- Notifications:
    - Users will not be on WebQuake all the time, but a disaster could happen at any moment.
    - To rectify this issue, WebQuake will send notifications to users who opt-in to them.
        - When an EEW or a tsunami warning is issued, a notification will be sent to users to warn them and prompt them to open WebQuake to get more information.
- Creating a way for users to personalise their experience:
    - Language switching (English/Japanese)
    - Notification control

2.1.2 Tree Diagram



2.2 Describe the solution

2.2.1 Explain and justify the structure of the solution

To create an online application, there must be two parts to the development:

- A backend, coded in Python, to receive, process and send data to the user.
- A frontend, coded with a combination of HTML, CSS, and JavaScript, which will receive the data sent by the backend and create an interface to view it.

Backend:

- A data receiver, to receive the data sent via WebSocket by Project DM-DSS.
    - A ping will need to be periodically sent to Project DM-DSS's servers so that the WebSocket does not time out and close.
- A data processor, to process the received data.
    - Distinct types and layouts of data will be sent depending on the type of information it is sending, so it must be able to recognise which data type is which and process it accordingly.
    - Data will only be sent in Japanese with its corresponding code, so a dictionary linking codes to their English translations will need to be created.
    - Tsunami warning heights will need to be interpreted into their corresponding warnings. For example, if the maximum estimated wave height is greater than 3 metres, it must be classified as a major tsunami warning.
    - A prediction algorithm for tsunamis.
    - Unknown value handling, so that the server does not crash if it comes across something unexpected.
- An SQLite server to store the processed data.
    - A multi-table layout will be used to split apart each data types. This is needed as different data types have different information and not every data type has every type of information included.
    - It will be written to and read from using the SQLite Python library.
    - SQLite is being used to reduce the complexity of the project; other database engines (like PostgreSQL) could be used to increase performance but would take longer to implement.
- Storing the raw received data:
    - Useful for debugging. I cannot perfectly simulate the data that WebQuake's server will receive and while the data processor will handle unknown values, they still cannot be processed until I am able to interpret it myself and edit the code to reflect this. Sample data found online does not cover every eventuality and is often multiple years old.
        - For example, tsunamis are a rare occurrence, so it is possible that my server will receive something it does not know how to interpret. Without storing the raw data it received, I will not be able to revise the extraction algorithm to be able to process the new data.
- A web server, to handle the sending and receiving of data to/from users:
    - Any time sensitive data will be sent using Websockets through Flask. This is to ensure that data arrives in a timely manner and in order. The data within the Websocket will be sent in JSON format, as this is the format the processed data will be in.

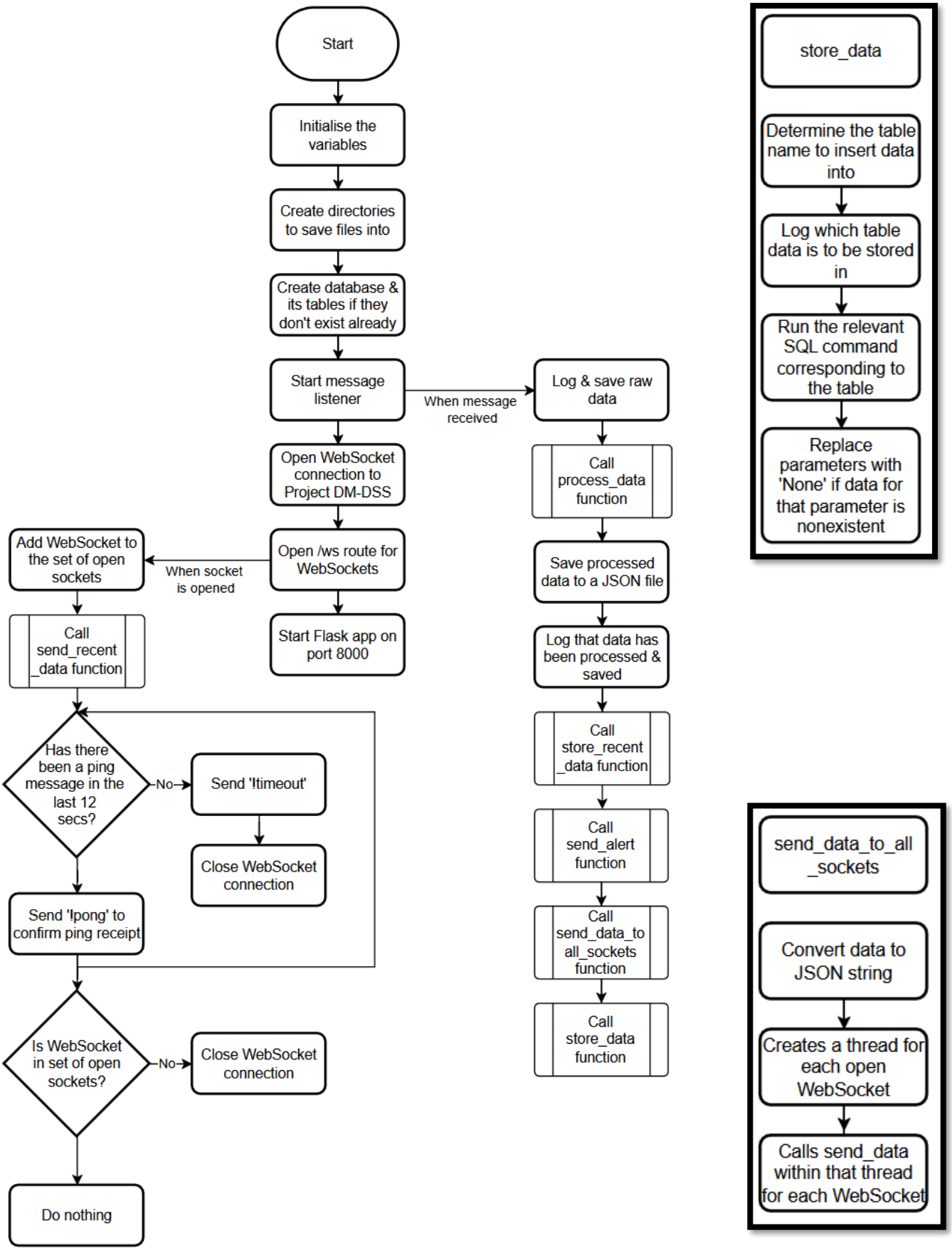
- Cloudflare Pages, to host any static HTML, CSS, and JavaScript.
    - During times of disaster, many people could be accessing WebQuake and therefore, WebQuake's server.
    - By offloading as much as possible to Cloudflare's Content Delivery Network, WebQuake's server is less likely to be overloaded when an EEW is issued and users open WebQuake.
    - Additionally, Cloudflare will provide WebQuake with free DDoS and bot protection. This will reduce the amount of bot traffic reaching WebQuake's server and protect it from attacks.
    - Cloudflare has datacentres in Japan. Having the user load most data from servers close to them decreases time spent waiting for WebQuake to load.
- OneSignal, to manage the distribution of notifications.
    - They provide a free to use API to register WebQuake's users to receive notifications and distribute web-push notifications for up to 10,000 subscribers for free.
    - Creating my own notification sender is possible but it is likely to be unreliable and resource heavy. Using OneSignal will reduce the complexity of the project and increase the reliability of notifications being delivered in a timely manner.
- A logger, to make fixing bugs easier.
    - Every time WebQuake receives, processes or stores data, it will log it to a file corresponding to when the server was started.

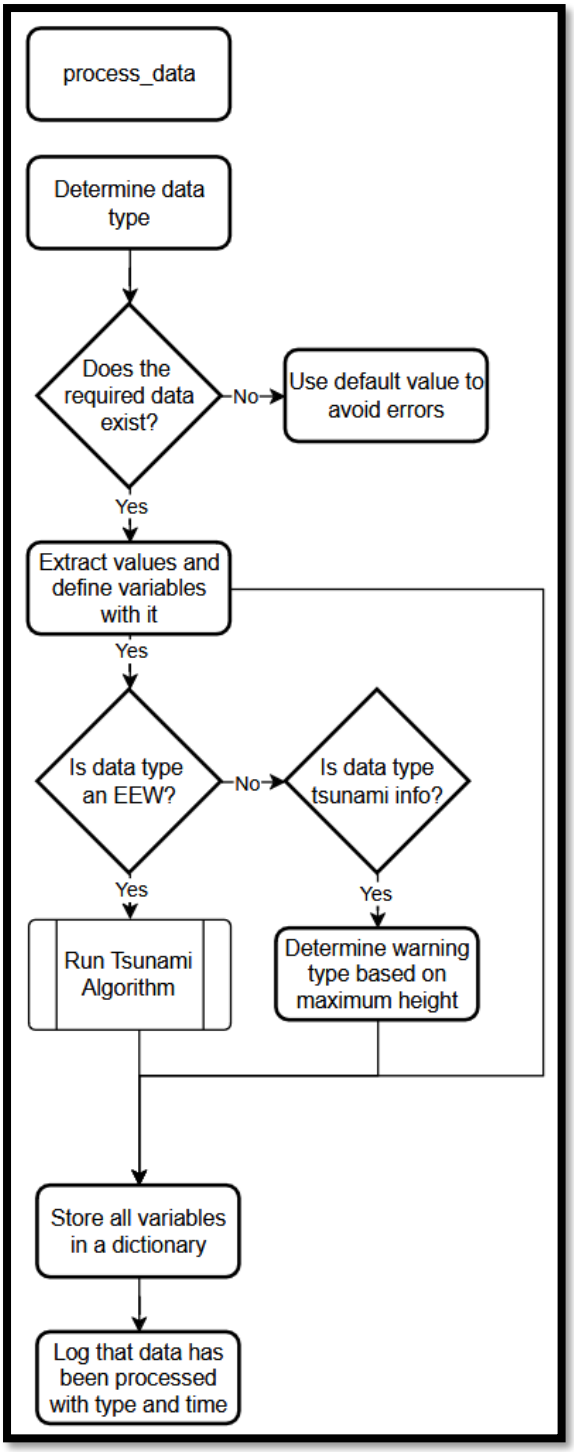
Frontend:

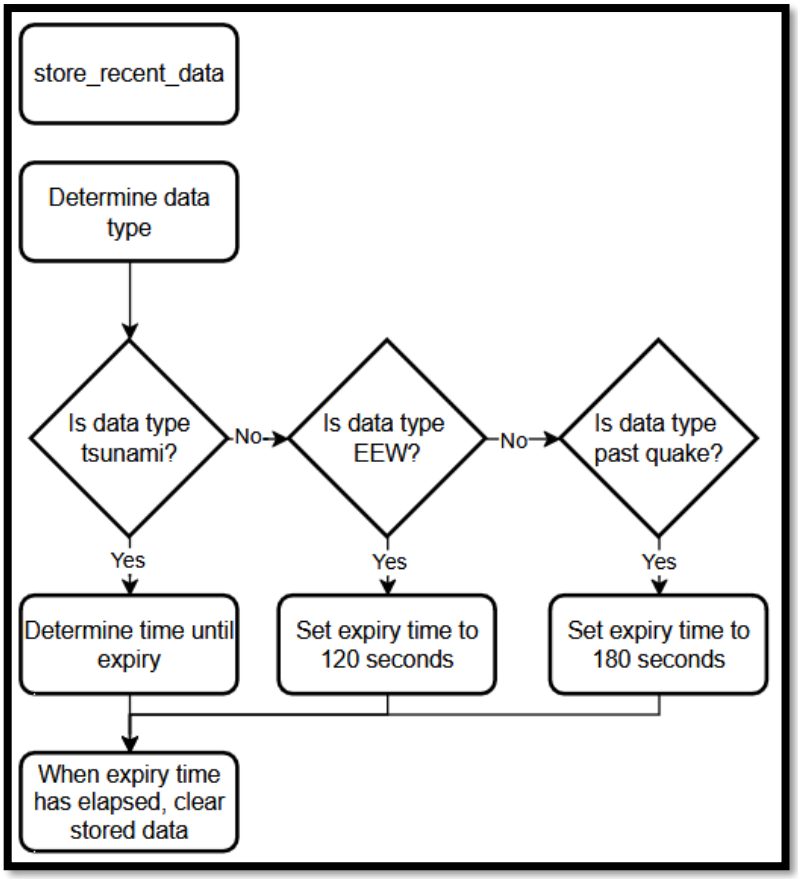
- HTML and CSS will be used to create a responsive user-friendly interface.
    - It must be able to keep its easy-to-use design across mobile and desktop platforms.
- A home page, containing:
    - Three tabs, one for EEWs, one for tsunami warnings and one for past earthquake information.
        - EEW tab must contain all relevant information (max est. I_{JMA}, depth, places issued for…)
        - Tsunami tab must contain where tsunami warnings are issued for, the type of warning and the estimated maximum heights.
            - Tsunami height observations will be viewable on the same tab.
        - Past earthquake information tab will contain any information relevant to that earthquake.
    - A top navbar, allowing tabs to be manually switched between.
- A settings page, to customise user's experience using WebQuake:
    - Language switcher (English/Japanese).
    - Notification toggles.
- JavaScript will be used for:
    - Receiving data from WebQuake's server.
        - Achieved via the use of a Websocket connection.
    - Process the received data into separate variables.
    - Change what is displayed based on received data.
    - Colour the est. I_{JMA}, est. LPGM and Tsunami Warning Level boxes based on their values.
    - Subscribing/unsubscribing from notifications.
    - Showing the user their connection health/status.
    - Switching between tabs.
    - Switching between languages.

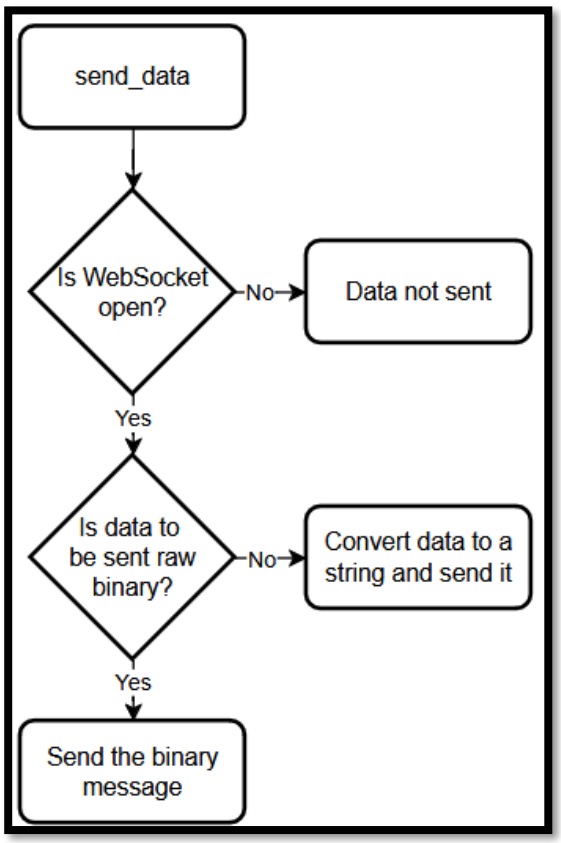
2.2.2 Describe the parts of the solution using algorithms

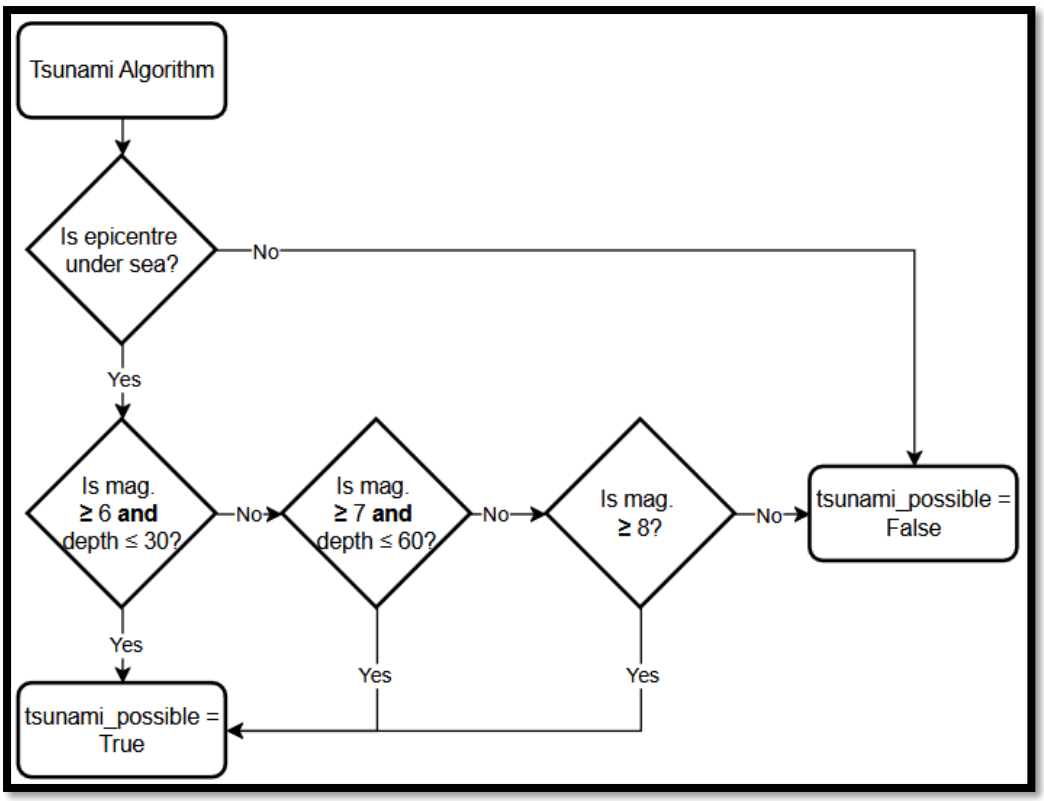
All functions called by the main thread have a black border surrounding them.

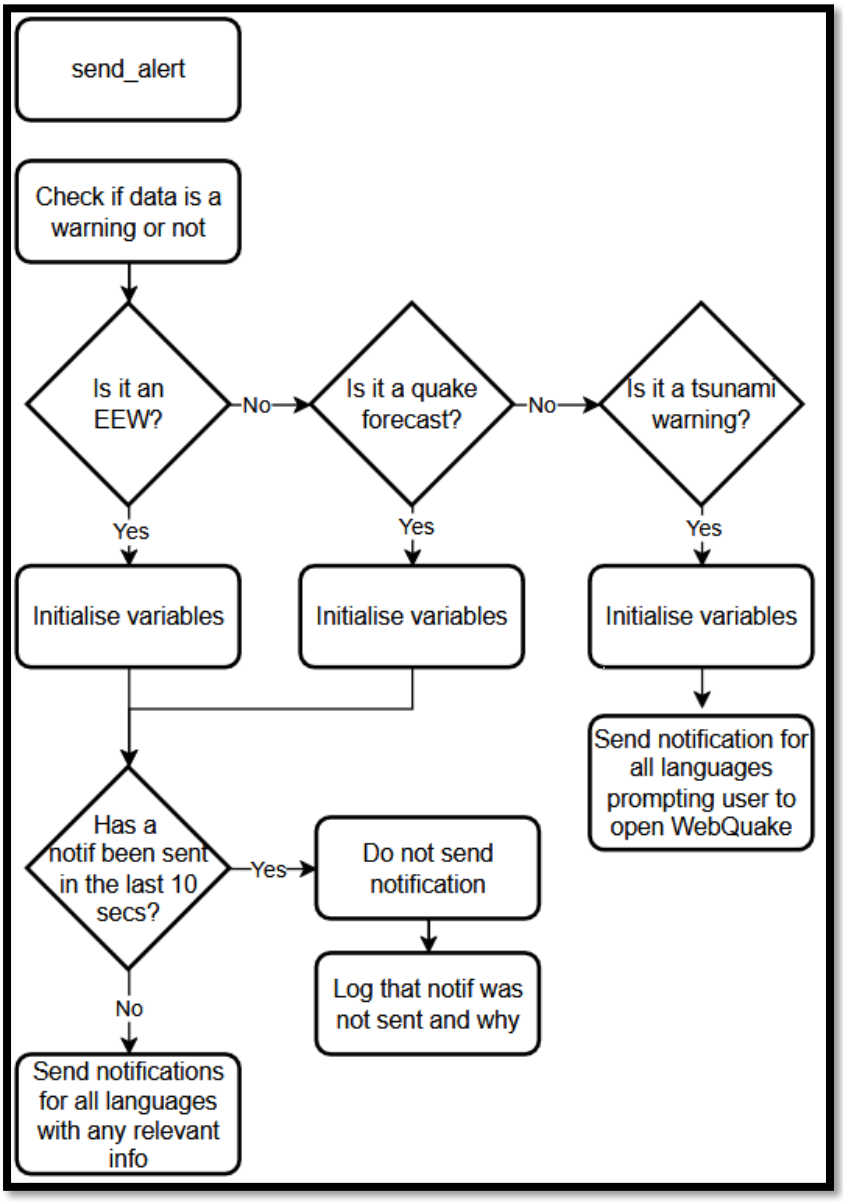



process_data
Determine data type
Does the required data exist?
No
Use default value to avoid errors
Yes
Extract values and define variables with it
Yes
Is data type an EEW?
No
Is data type tsunami info?
Yes
Yes
Run Tsunami Algorithm
Determine warning type based on maximum height
Store all variables in a dictionary
Log that data has been processed with type and time

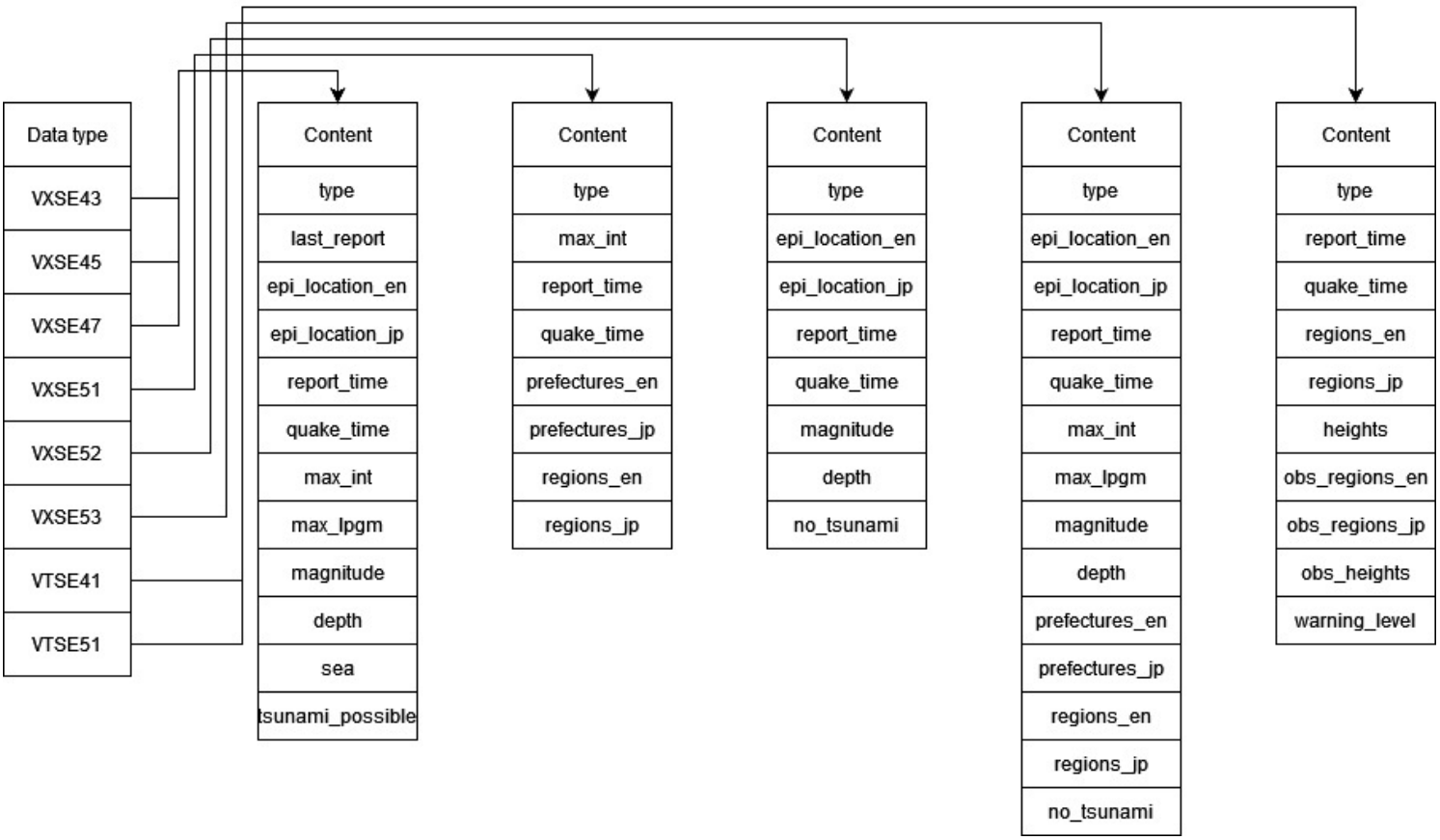

store_recent_data
Determine data type
Is data type tsunami?
No
Is data type EEW?
No
Is data type past quake?
Yes
Yes
Yes
Determine time until expiry
Set expiry time to 120 seconds
Set expiry time to 180 seconds
When expiry time has elapsed, clear stored data


send_data
Is WebSocket open?
No
Data not sent
Yes
Is data to be sent raw binary?
No
Convert data to a string and send it
Yes
Send the binary message


Tsunami Algorithm
Is epicentre under sea?
No
Yes
Is mag. ≥ 6 and depth ≤ 30?
No
Is mag. ≥ 7 and depth ≤ 60?
No
Is mag. ≥ 8?
No
tsunami_possible = False
Yes
Yes
Yes
tsunami_possible = True


send_alert
Check if data is a warning or not
Is it an EEW?
No
Is it a quake forecast?
No
Is it a tsunami warning?
Yes
Yes
Yes
Initialise variables
Initialise variables
Initialise variables
Send notification for all languages prompting user to open WebQuake
Has a notif been sent in the last 10 secs?
Yes
Do not send notification
Log that notif was not sent and why
No
Send notifications for all languages with any relevant info

2.2.3 Identify key variables/data structures/classes



2.2.4 File descriptions permanent storage

File name	output_data.db				
Access method	I will use Python to store and retrieve any data stored in the database.				
Table Name(s)	Column Name	Validation	Overview of validation rule applied	Description of field	Typical Data
VTSE41, VTSE51	report_time	Data type check	Must be Unix seconds	Time of report	1730748300
	quake_time			Time of quake	1730748300
	regions_en	Not validated		List of regions affected (English)	Tokyo Bay, Iyonada
	regions_jp			List of regions Affected (Japanese)	東京湾，伊予灘
	heights	Data type check	Must be an integer or a float	List of est. heights	0.5, 1, 3
	arrival_times	Not validated		List of times the tsunami is expected to arrive	1730748300, 1730749803
	obs_regions_en			List of regions with observations (Eng)	Tokyo Bay, Iyonada
	obs_regions_jp			List of regions with observations (Ja)	東京湾，伊予灘
	obs_heights	Data type check	Must be an integer or a float	List of observed heights	0.2, 5, 5.6
	warning_level	Not validated		Type of warning	Advisory
VXSE43, VXSE45, VXSE47	last_report	Data type check	Must be a boolean	If no more reports	True
	epi_location_en	Not validated		Epicentre location (English)	Tokyo Bay
	epi_location_jp			Epicentre location (Japanese)	東京湾
	report_time	Data type check	Must be Unix seconds	Time of report	1730748300
	report_num		Must be an integer	Report number	8
	quake_time		Must be Unix seconds	Time of quake	1730748300
	max_int		Must be an integer	Max. est. I_{JMA}	6
	max_lpgm			Max. est. LPGM	3
	magnitude		Must be an integer or float	Est. mag. of quake	4.7
	depth			Est. depth of quake	30
	sea		Must be a boolean	If epicentre is at sea	True
	warning			If warning is issued	False
	tsunami_possible			If tsunami is possible based on mag., depth & sea	False
VXSE51	max_int	Data type check	Must be an integer	Max. est. I_{JMA}	6
	report_time		Must be Unix seconds	Time of report	1730748300
	quake_time			Time of quake	1730748300
	prefectures_en	Not validated		List of prefectures affected (English)	Ishikawa, Okinawa
	prefectures_jp			List of prefectures affected (Japanese)	石川県, 沖縄県
	regions_en			List of regions affected (English)	Tokyo Bay, Iyonada
	regions_jp			List of regions Affected (Japanese)	東京湾，伊予灘

VXSE52	epi_location_en	Not validated		Epicentre location (English)	Tokyo Bay
	epi_location_jp			Epicentre location (Japanese)	東京湾
	report_time	Data type check	Must be Unix seconds	Time of report	1730748300
	quake_time			Time of quake	1730748300
	magnitude		Must be an integer or float	Est. mag. of quake	4.7
	depth			Est. depth of quake	30
	no_tsunami		Must be a boolean	Confirmation of no possible tsunami	True
VXSE53	epi_location_en	Not validated		Epicentre location (English)	Tokyo Bay
	epi_location_jp			Epicentre location (Japanese)	東京湾
	report_time	Data type check	Must be Unix seconds	Time of report	1730748300
	quake_time			Time of quake	1730748300
	max_int		Must be an integer	Max. est. I_{JMA}	6
	max_lpgm			Max. est. LPGM	3
	magnitude		Must be an integer or float	Est. mag. of quake	4.7
	depth			Est. depth of quake	30
	prefectures_en	Not validated		List of prefectures affected (English)	Ishikawa, Okinawa
	prefectures_jp			List of prefectures affected (Japanese)	石川県, 沖縄県
	regions_en			List of regions affected (English)	Tokyo Bay, Iyonada
	regions_jp			List of regions Affected (Japanese)	東京湾，伊予灘
	no_tsunami	Data type check	Must be a boolean	Confirmation of no possible tsunami	True

3 Developing the Solution

3.1 Iterative Development Process

3.1.1 Input Forms

Toggle Notifications

What does it do?

Checks if the user is subscribed to notifications or not. If not, it prompts the user to allow notifications, registers them with OneSignal and alerts the user that they have been successfully subscribed to notifications/it failed. If they are already registered, then it deregisters them from OneSignal and alerts the user that they have been successfully unsubscribed from notifications/it failed.

Code Snippets

```
function unsubscribeFromNotifications() {
    OneSignal.push(function () {
        OneSignal.setSubscription(false)
            .then(function () {
                console.log("User successfully unsubscribed from notifications.");
                if (currentLanguage === 'ja') {
                    alert("通知の購読を解除しました。");
                } else {
                    alert("You have been unsubscribed from notifications.");
                }
            })
            .catch(function (error) {
                console.error("Error unsubscribing from notifications:", error);
                if (currentLanguage === 'ja') {
                    alert("通知の購読解除に失敗しました。もう一度お試しください。");
                } else {
                    alert("Failed to unsubscribe from notifications. Please try again.");
                }
            });
    });
}
```

```
function enableNotifications() {
    OneSignal.push(function () {
        OneSignal.isPushNotificationsEnabled().then(function (isEnabled) {
            if (isEnabled) {
                if (currentLanguage === 'ja') {
                    alert("あなたはすでに通知を購読しています。");
                } else {
                    alert("You are already subscribed to notifications.");
                }
            } else {
                OneSignal.registerForPushNotifications()
                    .then(function () {
                        console.log("User subscribed to notifications.");
                        if (currentLanguage === 'ja') {
                            alert("通知が有効になりました！");
                        } else {
                            alert("Notifications have been enabled!");
                        }
                    })
                    .catch(function (error) {
                        console.error("Error subscribing to notifications:", error);
                        if (currentLanguage === 'ja') {
                            alert("通知を有効にできませんでした。ブラウザの設定を確認してください。");
                        } else {
                            alert("Failed to enable notifications. Please check your browser settings.");
                        }
                    });
            }
        });
    });
}
```

```
function toggleNotifications() {
    OneSignal.push(function () {
        OneSignal.isPushNotificationsEnabled().then(isEnabled => {
            if (isEnabled) {
                unsubscribeFromNotifications();
            } else {
                enableNotifications();
            }
        });
    });
}
```

Toggle EEW Subscription | Toggle Tsunami Warning Subscription | Toggle Earthquake Forecast Subscription

What does it do?

Checks if the user is subscribed to a specific notification tag or not. If not, sends a request to OneSignal to add that tag to their OneSignal user and alerts the user that they have been successfully subscribed to a specific notification/it failed. If they are already registered, it removes the tag from their OneSignal user and alerts the user that they have been successfully unsubscribed from a specific notification/it failed.

Code Snippets

```
function toggleNotifications() {
    OneSignal.push(function () {
        OneSignal.isPushNotificationsEnabled().then(isEnabled => {
            if (isEnabled) {
                unsubscribeFromNotifications();
            } else {
                enableNotifications();
            }
        });
    });
}
```

```
function removeTag(tagKey) {
    OneSignal.deleteTag(tagKey)
        .then(function() {
            console.log(`Tag '${tagKey}' removed successfully!`);
            if (currentLanguage === 'ja') {
                const translatedTag = tagKey.replace('_', ' ').replace('earthquake warnings', '地震警報の通知')
                .replace('tsunami warnings', '津波警報 通知').replace('earthquake forecasts', '地震動予測 通知');
                alert(`あなたは今、${translatedTag}を購読しています。`);
            } else {
                alert(`You are now unsubscribed from ${tagKey.replace('_', ' ')}.`);
            }
        })
        .catch(function(error) {
            console.error(`Failed to remove tag '${tagKey}':`, error);
            if (currentLanguage === 'ja') {
                const translatedTag = tagKey.replace('_', ' ').replace('earthquake warnings', '地震警報の通知')
                .replace('tsunami warnings', '津波警報 通知').replace('earthquake forecasts', '地震動予測 通知');
                alert(`${translatedTag}の購読解除に失敗しました。`);
            } else {
                alert(`Failed to unsubscribe from ${tagKey.replace('_', ' ')}.`);
            }
        });
}
```

```
function toggleTag(tagKey) {
    let adjustedTag = tagKey;
    if (currentLanguage === 'ja') {
        adjustedTag = tagKey + '_ja';
    }

    OneSignal.push(function () {
        OneSignal.getTags().then(tags => {
            if (tags[adjustedTag] === "subscribed") {
                removeTag(adjustedTag);
            } else {
                addTag(adjustedTag);
            }
        });
    });
}
```

```
function addTag(tagKey) {
    OneSignal.sendTag(tagKey, "subscribed")
        .then(function() {
            console.log(`Tag '${tagKey}' added successfully!`);
            if (currentLanguage === 'ja') {
                const translatedTag = tagKey.replace('_', ' ').replace('earthquake warnings', '地震警報の通知')
                .replace('tsunami warnings', '津波警報 通知').replace('earthquake forecasts', '地震動予測 通知');
                alert(`あなたは今、${translatedTag}を購読しています。`);
            } else {
                alert(`You are now subscribed to ${tagKey.replace('_', ' ')}.`);
            }
        })
        .catch(function(error) {
            console.error(`Failed to add tag '${tagKey}':`, error);
            if (currentLanguage === 'ja') {
                const translatedTag = tagKey.replace('_', ' ').replace('earthquake warnings', '地震警報の通知')
                .replace('tsunami warnings', '津波警報 通知').replace('earthquake forecasts', '地震動予測 通知');
                alert(`${translatedTag}の購読に失敗しました。`);
            } else {
                alert(`Failed to subscribe to ${tagKey.replace('_', ' ')}.`);
            }
        });
}
```

3.1.2 Writing to files

The SQLite database is stored as a single file called “output_data.db” which is read from and written to using Python’s inbuilt SQLite library. Data is saved to the table after it has been processed.

Every time it receives data, the server sorts and saves the file into a directory called “data_messages.” Inside that folder are more folders which further sort the messages into their respective types and the files are given names corresponding to when they were saved.

Every time it processes data, the server saves the file into a directory called “decoded_messages” and the files are given names in Unix time according to when they were saved.

The server logs when it completes or fails to complete an action, to make debugging easier. If the server fails to process some data, I can read through the log to figure out what data it could not process and fix the issue.

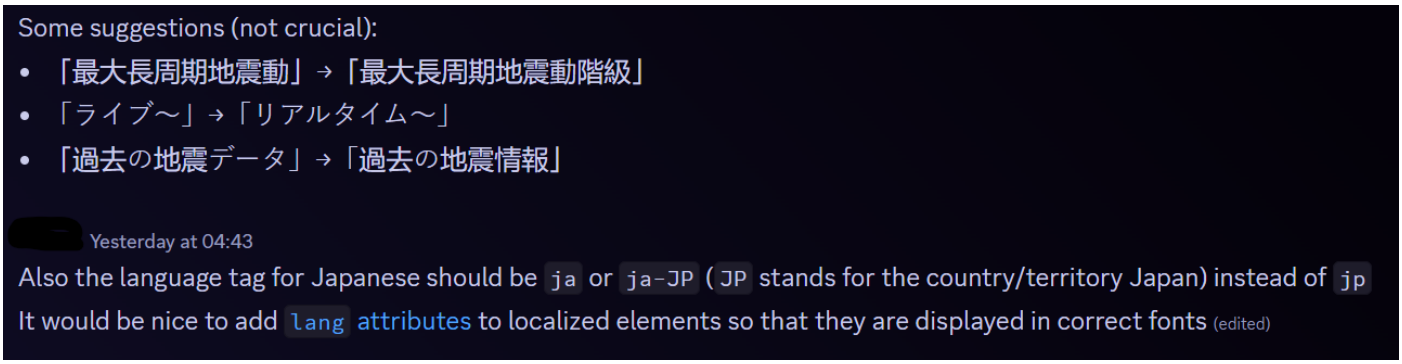
3.1.3 Processing data

Processing data into the database is discussed in 2.2.4 but processing also happens when the client receives the data from the server. For example:

Data Received	Data Processing	Sample Output	Reason
“quake_time.en”=1734634620	Converting Unix seconds into ISO date/time format	2024/12/25, 01:02:03	Humans cannot read Unix seconds
“final_report_jp”=True	Translating Booleans to Japanese	はい	Japanese speakers may not understand “True” or “False”
“regions_en”=“Tokyo” “heights_en”=2	Matching up regions with their heights	Estimated Wave Heights: Tokyo, 2	It may take a while to access info relevant for you if you must match up yourself.

3.1.4 Output

The user will pick their language at the top and when a different language is selected, everything on screen (apart from WebQuake) is translated locally into that language. This is achieved by having different elements for English and Japanese and hiding one or the other when languages are selected.
I asked some friends of mine (who speak Japanese) to go over the translations for me and any suggestions they had, I implemented.



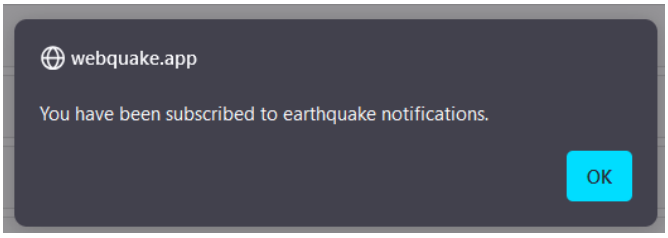


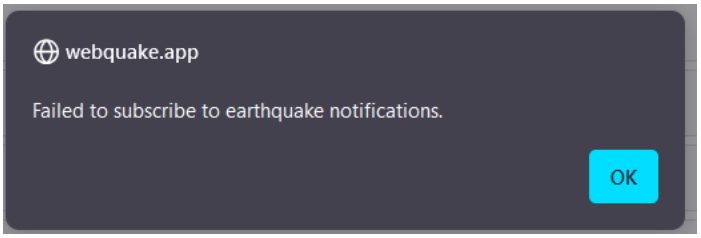
With this, I was able to make sure my translations were accurate and easily readable by those who can read Japanese. Any extra time that may have been spent by the user interpreting an unclear meaning from a bad translation can now be used to act on the information WebQuake gives them.

There will be a choice of three tabs, and the user may manually select a tab from the top of the page. However, upon an event occurring, the interface will automatically switch to the tab with information.
All information is collated by the client and sorted into boxes within those tabs:

- Live Earthquake Warnings/リアルタイム地震警報:
  - Box titles (English): EEW issued, Report Number, Epicentre Location, Quake Time, Max Intensity, Max LPGM, Magnitude, Depth, Prefectures under warning, Final Report, Epicentre under sea, Tsunami Possible
  - Box titles (Japanese): 緊急地震速報の発表, レポート番号, 震源地, 地震発生時刻, 報告時間, 最大震度, 最大長周期地震動階級, マグニチュード, 深さ, 警報中の都道府県, 最終報告, 海底震源, 津波の可能性
- Live Tsunami Warnings/リアルタイム津波警報:
  - Box titles (English): Quake Time, Regions, Estimated Wave Heights, Observed Wave Heights, Warning Level
  - Box titles (Japanese): 地震発生時刻, 警告が出される地域, 推定波高, 観測波高, 警告レベル
- Past Earthquake Data/過去の地震情報:
  - Box titles (English): Epicentre Location, Quake Time, Max Intensity, Max LPGM, Magnitude, Depth, Prefectures, No Tsunami
  - Box titles (Japanese): 震源地, 地震発生時刻, 最大震度, 最大長周期地震動階級, マグニチュード, 深さ, 警報中の都道府県, 津波なし

When a user clicks on a button to subscribe to notifications, upon confirmation from OneSignal that their subscription is registered, the user will be alerted (in their selected language) that their subscription has been successful/unsuccessful.





At the bottom of the page, the current time (JST) will be displayed. This is useful for comparing when an event happened to the time now, and for seeing the delay between reality and the user's connection to WebQuake's server.
Also near the bottom of the page will be a bar showing the user's connection health.

- Data received at least every 1.5 seconds: "Connection Stable," Colour: Green
- Every 1.5-10 seconds: "Connection Unstable", Colour: Orange
- Nothing for >10 seconds: "Reconnecting... Attempt no:", Colour: Orange
  - Five connection attempts will be tried over 25 seconds.
- Five failed connection attempts: "Connection lost. Please refresh the page.," Colour: Red



A screenshot showing the Live Tsunami Warnings tab with **example** data in both English and Japanese

4 Testing to Inform Development

As I progress through the development of my solution, I will inevitably come across difficulties. These tables explain the difficulties I saw and how I overcame them. Green highlights are code I have added and red highlights are code which was removed.

Task Description	Data	Expected Result	Actual Result
Parsing the data received into JSON format	‘!pong’	!pong would be ignored as it is not in JSON format	!pong could not be parsed as it was not JSON data

How I fixed the encountered error

```
Error parsing JSON: SyntaxError: JSON.parse: unexpected character at line 1 column 1 of the JSON data
```

I added a check to my createWebSocket function to skip parsing the data if the received data was exactly ‘!pong’

```
socket.addEventListener('message', (event) => {
    if (event.data.trim() === '!pong') {
        return;
    }
    try {
        const data = JSON.parse(event.data);
        displayData(data);
```

Task Description	Data	Expected Result	Actual Result
Duplicating data which does not need to be translated	‘max_int’ = 3	‘max_int_en’ = 3 ‘max_int_jp’ = 3	‘max_int’ = 3

How I fixed the encountered error

I reworked some of the logic for deciding whether to duplicate the data or not and created the new data under a different title.

```
                const data = JSON.parse(event.data);
                for (const key in data) {
                    if (data.hasOwnProperty(key) && !key.endsWith('_en') && !key.endsWith('_jp')) {
                        data[`${key}_en`] = data[key];
                        data[`${key}_jp`] = data[key];
                    }
                }
                displayData(data);
            } catch (error) {
                console.error('Error parsing JSON:', error);
            }
        });
```

```
                const data = JSON.parse(event.data);
+               const updatedData = { ...data };
+
+               // Duplicate data for `_en` and `_jp` keys if they are missing
                for (const key in data) {
+                   if (data.hasOwnProperty(key) && (!key.endsWith('_en') || !key.endsWith('_jp'))) {
+                       updatedData[`${key}_en`] = data[key];
+                       updatedData[`${key}_jp`] = data[key];
                    }
                }
+               displayData(updatedData);
            } catch (error) {
                console.error('Error parsing JSON:', error);
            }
        });
```

Task Description	Data	Expected Result	Actual Result
Colouring boxes based on their value	‘max_lpgm’ = 2	`<div class=“lpgm-2 data-item en”…>Max LPGM: 2</div>`  and the same for Japanese	`<div class=“data-item en”…>Max LPGM: 2</div>`  and the same for Japanese

How I fixed the encountered error

I made sure it was applying the change for when the key had a suffix of either ‘_en’ or ‘_jp,’ not both. Also, I changed the location of the new class to be added to the start, so that the properties of “data-item” did not override the new colour changes. The same fix was applied over maximum intensity and tsunami warning type colouring as well.

```
    if (key === 'max_lpgm_en' && key === 'max_lpgm_jp') {                    |  432                 if (key === 'max_lpgm_en' || key === 'max_lpgm_jp') {
@@ -451,7 @@                                                                |
        }                                                                    |  451                 }
                                                                             |  452
        items.forEach(item => {                                              |  453                 items.forEach(item => {
            item.classList.add(lpgmClass);                                   |  454 +                   item.className = `${lpgmClass} ${item.className}`
        });                                                                  |  455                 });
    }                                                                        |  456             }
}                                                                            |  457         }
```

Task Description	Data	Expected Result	Actual Result
Subscribing the user to receive notifications on button press.	N/A	User is subscribed to notifications with an alert to confirm.	User was not subscribed to notifications with no alert to inform the user that the action had failed.

How I fixed the encountered error

I made sure that OneSignal was initialised after the page had loaded fully, to ensure that the buttons and their respective functions were able to communicate with OneSignal. I also added an error in console which appears when OneSignal has not loaded correctly, to help the user identify the issue.

```
OneSignal.push(function() {                                                  |  67  +   window.onload = function() {
  OneSignal.init({                                                           |  68  +     if (typeof OneSignal !== 'undefined') {
    appId: "9e4a84de-c3f9-4e84-b285-f1601c5e37de",                           |  69  +       OneSignal.push(function() {
    notifyButton: {                                                          |  70  +         OneSignal.init({
      enable: true,                                                          |  71  +           appId: "9e4a84de-c3f9-4e84-b285-f1601c5e37de"
    },                                                                       |  72  +           notifyButton: {
  });                                                                        |  73  +             enable: true,
                                                                             |  74  +           },
                                                                             |  75  +         });
                                                                             |  76
  console.log("OneSignal initialized!");                                     |  77  +       console.log("OneSignal initialised!");
                                                                             |  78
  // Function to assign a tag                                                |  79      // Function to assign a tag
  function addTag(tagKey) {                                                  |  80      function addTag(tagKey) {
@@ -131,7 +133,11 @@                                                            |
      removeTag('earthquake_forecasts');                                     |  133           removeTag('earthquake_forecasts');
    });                                                                      |  134       });
  });                                                                        |  135     });
});                                                                          |  136 +   });
                                                                             |  137 + } else {
                                                                             |  138 +   console.error("OneSignal SDK is not loaded properly.")
                                                                             |  139 + }
                                                                             |  140 + };
```

5 Testing to inform evaluation

5.1 Testing to inform function and robustness

5.1.1 Calculation Test Plan

Calculation tested	Data	Purpose	Expected outcome	Notes
Tsunami algorithm	magnitude: 6.8, depth: 30, sea = True	To determine whether a tsunami is possible based on the data given	If the parameters are met, it sets a variable called tsunami_possible to True	1.a.iiv. Uses an algorithm to determine whether a tsunami is possible and tells the user. Tsunami warnings are usually issued within 3 minutes but if the epicentre is very close to land, the extra notice this can give a user could be crucial to escaping the extreme waves.

All parameters are casted to their required types to ensure processing does not fail.

If the epicentre isn't under the sea, a tsunami is extremely unlikely.

Below this magnitude, a tsunami is very unlikely.

Severity of the tsunami is linked to both magnitude and depth. Generally, the deeper the quake, the more magnitude required to trigger a tsunami.

The JMA's EEW system struggles to predict magnitudes of >8. [17] If above this is predicted, inaccuracies are common, so this will err on the side of caution.

```
if bool(sea) == True and float(magnitude) >= 6.0 and float(depth) <= 30 or float(magnitude) >= 7.0 and float(depth) <= 60 or float(magnitude) >= 8.0
    tsunami_possible = True
else:
    tsunami_possible = False
```

Calculation tested	Data	Purpose	Expected outcome	Notes
Time conversion	quake_time: 1736780387	To convert Unix timestamps into human readable time	Converts to DD/MM/YYYY HH:MM:SS	Generally, people don't see a Unix timestamp as a time & date, they just see it as a number. Converting it to a widely readable format makes sure there are no discrepancies.

Converted from Unix seconds to Unix milliseconds.

Ensures that the format is the same as the commonly used one in the user's region.

```
if (data.quake_time) {
    const quakeDate = new Date(data.quake_time * 1000);
    data.quake_time_en = quakeDate.toLocaleString('en-GB', { timeZone: 'Asia/Tokyo' });
    data.quake_time_jp = quakeDate.toLocaleString('ja-JP', { timeZone: 'Asia/Tokyo' });
}
```

Ensures that the time is displayed in JST.

5.1.2 Validation Test Plan

Tests using Valid Data

Component	Data	Reason for choice of data	Expected outcome
heights	5, 5, 3, 1.5, 1, 0.5	Valid Data Type: Float (list)	Data is accepted and added to database
obs_heights	4.6, 3.8, 1.5, 0.8, 0.2	Valid Data Type: Float (list)	Data is accepted and added to database
last_report	True	Valid Data Type: Boolean	Data is accepted and added to database
report_time	1736780387	Valid Data Type: Unix seconds	Data is accepted and added to database
quake_time	1736780387	Valid Data Type: Unix seconds	Data is accepted and added to database
max_int	7	Valid Data Type: Integer	Data is accepted and added to database
max_lpgm	3	Valid Data Type: Integer	Data is accepted and added to database
magnitude	6.8	Valid Data Type: Float	Data is accepted and added to database
depth	30	Valid Data Type: Integer	Data is accepted and added to database
sea	True	Valid Data Type: Boolean	Data is accepted and added to database
warning	True	Valid Data Type: Boolean	Data is accepted and added to database
tsunami_possible	True	Valid Data Type: Boolean	Data is accepted and added to database
no_tsunami	False	Valid Data Type: Boolean	Data is accepted and added to database

Extreme Data to test Lower Bound

Component	Data	Reason for choice of data	Expected outcome
heights	0.2, 0.2	Extreme Valid Lower Bound Type: Float (list)	Data is accepted and added to database
obs_heights	0.1, 0.1	Extreme Valid Lower Bound Type: Float (list)	Data is accepted and added to database
last_report	False	Extreme Valid Lower Bound Type: Boolean	Data is accepted and added to database
report_time	0	Extreme Valid Lower Bound Type: Unix seconds	Data is accepted and added to database
quake_time	0	Extreme Valid Lower Bound Type: Unix seconds	Data is accepted and added to database
max_int	0	Extreme Valid Lower Bound Type: Integer	Data is accepted and added to database
max_lpgm	0	Extreme Valid Lower Bound Type: Integer	Data is accepted and added to database
magnitude	0	Extreme Valid Lower Bound Type: Float	Data is accepted and added to database
depth	0	Extreme Valid Lower Bound Type: Integer	Data is accepted and added to database
sea	False	Extreme Valid Lower Bound Type: Boolean	Data is accepted and added to database
warning	False	Extreme Valid Lower Bound Type: Boolean	Data is accepted and added to database
tsunami_possible	False	Extreme Valid Lower Bound Type: Boolean	Data is accepted and added to database
no_tsunami	False	Extreme Valid Lower Bound Type: Boolean	Data is accepted and added to database

Extreme Data to test Higher Bound

Component	Data	Reason for choice of data	Expected outcome
heights	15, 10, 10	Extreme Valid Upper Bound Type: Float (list)	Data is accepted and added to database
obs_heights	16.4, 15.2	Extreme Valid Upper Bound Type: Float (list)	Data is accepted and added to database
last_report	True	Extreme Valid Upper Bound Type: Boolean	Data is accepted and added to database
report_time	2147483647	Extreme Valid Upper Bound Type: Unix seconds	Data is accepted and added to database
quake_time	2147483647	Extreme Valid Upper Bound Type: Unix seconds	Data is accepted and added to database
max_int	7	Extreme Valid Upper Bound Type: Integer	Data is accepted and added to database
max_lpgm	4	Extreme Valid Upper Bound Type: Integer	Data is accepted and added to database
magnitude	10	Extreme Valid Upper Bound Type: Float	Data is accepted and added to database
depth	750	Extreme Valid Upper Bound Type: Integer	Data is accepted and added to database
sea	True	Extreme Valid Upper Bound Type: Boolean	Data is accepted and added to database
warning	True	Extreme Valid Upper Bound Type: Boolean	Data is accepted and added to database
tsunami_possible	True	Extreme Valid Upper Bound Type: Boolean	Data is accepted and added to database
no_tsunami	True	Extreme Valid Upper Bound Type: Boolean	Data is accepted and added to database

Invalid Data

Component	Data	Reason for choice of data	Expected outcome
heights	fifteen	Invalid Data Type: Float (list)	Data is rejected and not stored in database as data is not a float
obs_heights	fourteen	Invalid Data Type: Float (list)	Data is rejected and not stored in database as data is not a float
last_report	Yes	Invalid Data Type: Boolean	Data is rejected and not stored in database as data is a string, not a boolean
report_time	9999999999	Invalid Data Type: Unix seconds	Data is rejected and not stored in database as the max. for a Unix timestamp is 2147483647
quake_time	9999999999	Invalid Data Type: Unix seconds	Data is rejected and not stored in database as
max_int	five hundred	Invalid Data Type: Integer	Data is rejected and not stored in database as the max. for a Unix timestamp is 2147483647
max_lpgm	one point five	Invalid Data Type: Integer	Data is rejected and not stored in database as data is a string, not an integer
magnitude	?	Invalid Data Type: Float	Data is rejected and not stored in database as '?' is not a float
depth	too high	Invalid Data Type: Integer	Data is rejected and not stored in database as data is not an integer
sea	No	Invalid Data Type: Boolean	Data is rejected and not stored in database as a boolean must be True or False
warning	1x8bnkw0	Invalid Data Type: Boolean	Data is rejected and not stored in database as data is a string, not a boolean
tsunami_possible	yeah	Invalid Data Type: Boolean	Data is rejected and not stored in database as 'yeah' is not a boolean
no_tsunami	!!	Invalid Data Type: Boolean	Data is rejected and not stored in database as '!!' is not a boolean

6 Evaluation

6.1 Success of the solution

Success criteria	Fully/Partially/ Not met	Explanation
Creates an interface to view data	Fully	Quite a broad criterion but an interface is created by WebQuake to view the data.
Shows current date/time (JST)	Fully	The current date & time are shown at the bottom of the page. "Time (JST)" is translated as needed.
Interprets NIED gif into I_{JMA} and displays observed values to user	Not	Outside the scope of the project – would have taken too much time to develop and implement. Also, it is implicitly forbidden in NIED's terms of use. [18]
Dynamically focuses on the most important information	Partially	Information is always switched to when data is received but there is no priority system. The priority system was deemed to be not hugely beneficial over the most recent information as importance of data is subjective.
Shows Earthquake Early Warnings	Fully	Shown under 'Live Earthquake Warnings' tab. Differentiating between a warning and not a warning is shown under 'EEW issued:'.
Shows location, magnitude and depth of EEW	Fully	Shown under 'Live Earthquake Warnings' tab.
Shows est. I_{JMA} per prefecture	Partially	Shown under 'Live Earthquake Warnings' tab. Prefectures under warning and est. I_{JMA} are shown separately.
Shows P/S wave of earthquake	Not	Would have required a map interface and a complicated algorithm to determine position of the wave. It would have been very challenging to produce an accurate representation so was considered outside the scope of the project.
Shows EEW report number	Fully	Shown under 'Live Earthquake Warnings' tab.
Shows estimated LPGM	Fully	Shown under 'Live Earthquake Warnings' tab. Box is coloured according to the number and the colours used by the JMA.
Shows tsunami warnings and their type	Fully	Shown under 'Live Tsunami Warnings' tab. Information is split up into boxes and the type of warning is coloured according to the colours used by the JMA.
Shows est. wave heights per prefecture	Fully	Shown under 'Live Tsunami Warnings' tab. Wave heights are grouped with their corresponding region.
Shows est. arrival times per prefecture	Fully	Shown under 'Live Tsunami Warnings' tab. Arrival times are grouped with their corresponding region.
Shows observed wave heights per prefecture	Fully	Shown under 'Live Tsunami Warnings' tab. Wave heights are grouped with their corresponding region.
Shows official earthquake reports	Fully	Shown under 'Past Earthquake Data' tab. Relevant information is split up into boxes.
Shows location, magnitude, intensity and depth of report	Fully	Shown under 'Past Earthquake Data' tab. Location is translated to English using a dictionary and intensity is coloured according to colours used by the JMA.

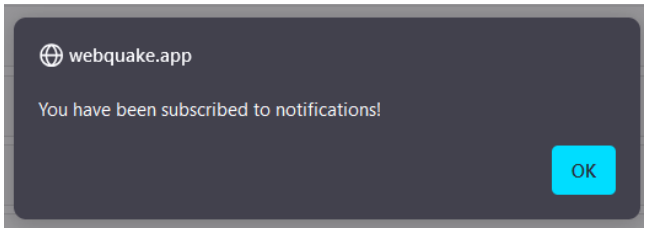
Shows time & date of report	Fully	Shown under 'Past Earthquake Data' tab. Converted from Unix timestamp to readable time/date format.
Shows LPGM	Fully	Shown under 'Past Earthquake Data' tab. Coloured according to colours used by the JMA.
Allows users to subscribe to notifications	Fully	Users can toggle their notification permission with a button at the bottom. On first press of the button, their browser will prompt them to allow notifications. After confirmation, the user is alerted to their successful subscription or told it failed.
Allows users to choose what notifications to subscribe to	Fully	Achieved using OneSignal 'tags'. Users can toggle which notifications they wish to subscribe to with buttons at the bottom. Does not require browser prompting. Users will be alerted to a successful tag addition or told it failed.
Gives notifications for EEWs	Fully	The backend sends a request to OneSignal's API to send out the notification with relevant information.
Gives notifications for observations of I_{JMA} >= 5-	Not	Would require NIED gif processing, so is not possible.
Gives notifications for tsunami warnings & advisories	Fully	The backend sends a request to OneSignal's API to send out the notification. The notification will prompt users to open WebQuake to view more information as there is too much information to cram into a notification.

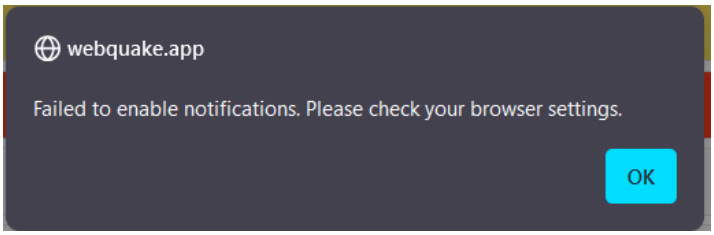
6.2 Describe the final product



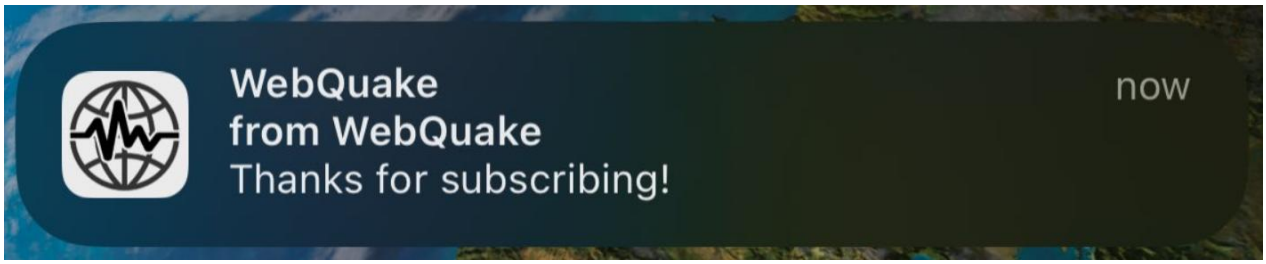
Toggle Notifications | Toggle EEW Subscription | Toggle Tsunami Warning Subscription | Toggle Earthquake Forecast Subscription

Upon clicking a notification toggle, users are alerted to the operation being a success or a failure





Further confirmation is sent to the user when notifications are successfully enabled so that the user has no doubt that notifications are working



Example of a real-life notification sent to my iPhone. When I clicked on it, I was brought to the correct information on WebQuake.



6.3 Maintenance and development

6.3.1 The maintainability of the solution

WebQuake should be fairly simple to keep running, only occasional light maintenance will be required (at least in the near future).

One concern for WebQuake's maintainability is a potential to overload the server. This would not be caused by incoming data, as data is never sent frequently enough to do this, but rather a sudden influx of users when a major event occurs. Serving the site is not an issue, Cloudflare does that for me, but every time someone opens WebQuake, a websocket must be opened. There will be a limit to how many websockets WebQuake's server can have open at once and if exceeded, the server may slow down or stop completely. I have tested WebQuake ability to handle floods of users using Loadster, with 4000 users accessing the site over 2 minutes. I monitored the server's health using Netdata while this was happening, and I could see no issues. The CPU utilisation never crept over 10% and network bandwidth was far from its limit (0.5% of the server's theoretical limit). However, synthetic tests can never perfectly replicate real life and the potential to overload WebQuake's server is still a concern.

Another concern is the storage space required to log and store everything needed for WebQuake. This wouldn't be a problem for a while, as WebQuake does not store much data and what is stored is text, but it is possible that WebQuake could run out of room to write to the database and create new files. This would cause errors when it attempts to store files. Writing new data to the database, creating log files and writing raw data would not be possible. Furthermore, operating systems tend to crash, occasionally into an unrecoverable state when they have no storage left. In the worst-case scenario, all of WebQuake's past data would be lost. The code would not be lost as it is backed up to GitHub.

It is possible that the JMA will change the schema used to send their data, in the future. This is likely to severely affect the processing of data as WebQuake will not know where to look to find the data it needs. Human intervention will be required to comb through the new data and tell WebQuake where the new place to look for data is. Additionally, this may affect database storage as the JMA could change how they send the data itself.

At some point in the future, the current methods and algorithms that the JMA uses to predict earthquakes and tsunamis will almost certainly be replaced by improved models. Following this, the JMA are likely to cease the sending of the old data, as it is now redundant. If this happened, a complete recode of how WebQuake processes data will be required, along with a new database. Migrating the old data to the new database is likely to be challenging as merging old data with new often leads to inaccuracies.

All tasks are split into functions, meaning that should any new features need to be added, current code shouldn't need to be heavily edited, and new functions can build on the old ones.

6.3.2 Further development of the solution

If I were able to develop WebQuake further, I would like to complete the map interface. This would greatly increase useability as it can create a visual representation of what you need to understand which makes it easier to understand for the user, as well as more functional. I would also like to add a way to go back in time and view events as they happened, with a timeline or something similar. This would help people in the short term after a natural disaster as they can go back and see exactly what happened and choose how to act based on that information, and it could also help people in the long-term as they would be able to go back and view exactly what to expect and how to correctly interpret the information in future times of disaster. The final thing I'd like to do is make WebQuake into a mobile application, without the use of PWAs or requiring the user to use a web browser. This would make the experience more seamless for mobile users, while opening opportunities to more features, like Apple's Live Activities.

Glossary:

NHK – Nippon Hōsō Kyōkai (a public Japanese broadcaster funded by license fees similar to the BBC).
NIED – The National Research Institute for Earth Science and Disaster Resilience
JMA – Japan Meteorological Agency
Seismic intensity/shindo/I_{JMA} – a scale (1 to 7) to measure shaking due to earthquakes made by the JMA.
Earthquake Reports – reports made by the JMA for the purpose of informing about a currently occurring earthquake.
MMI – Modified Mercalli Index, a scale (I to XII) to measure the intensity of shaking due to earthquakes.
PGA – Peak Ground Acceleration, a measure of earth shaking at a given geographical point.
Project DM-DSS – Project Disaster Mitigation – Data Send Service
Seismic points – a network of >600 motion sensors run by the NIED.
Major Tsunami Warning – an emergency warning sent by the JMA when tsunami waves of >3m are likely (alerts sent to phones, emergency broadcasting on TV and radio stations run by NHK).
Tsunami Warning – an emergency warning sent by the JMA when tsunami waves of 1-3m are likely (alerts sent to phones, emergency broadcasting on TV and radio stations run by NHK).
Tsunami Advisory – an advisory sent by the JMA when tsunami waves of 0.2-1m are likely (alerts sent to phones, broadcasted on TV and radio stations run by NHK but not as an emergency broadcast).
Tsunami Forecast – an advisory sent by the JMA when tsunami waves of <0.2m are likely (not broadcasted).
Interim earthquake report – a report made by the JMA when there was a shindo observation of 3+ which shows the intensities that were felt in each prefecture but not the estimated epicentre or magnitude.
Tsunami Observation – data relayed from observation points along the coast which indicates the real current height of the tsunami. Sometimes given as “到達” (reached) when the observation is significantly less than the estimated height so that a false sense of security is not created.
Earthquake Early Warning/EEW – an emergency warning sent by the JMA when an earthquake occurs which is estimated to cause shaking of >shindo 5-.
P-wave – the first and least damaging wave of shaking for an earthquake. Used to deliver EEWs before the s-wave occurs.
S-wave – the last and most damaging wave of shaking for an earthquake.
Conventional (method used to give warning) – A method of estimating seismic intensity, epicentre location, magnitude, and depth of an earthquake by using seismic wave data captured by seismometers.
PLUM – Propagation of Local Undamped Motion. A method of estimating seismic intensity directly from observed intensity without estimating the hypocentre & scale.
Depth (of an earthquake) – how far below ground level an earthquake occurs.
Long Period Ground Motion/LPGM – any seismic wave with a period of <1Hz. Particularly dangerous for high-rise buildings as frequencies of <1Hz are found to resonate with them.
LPGM Classes (1-4) – the classification of LPGM by the JMA under 4 classes. Particularly relevant for estimating the effects of LPGM in high-rise buildings. The higher the class, the more difficult it is to remain standing and the greater the expected building damage.

References:

[1] - https://www.nhk.or.jp/strl/english/publica/bt/41/19.html
[2] - https://www.statista.com/statistics/275102/share-of-the-population-to-own-a-smartphone-japan/
[3] - https://www.statista.com/statistics/643765/japan-monthly-working-hours/
[4] - https://www.gfdrr.org/sites/default/files/publication/knowledge-note-japan-earthquake-2-5_0.pdf
[5] - https://www.statista.com/statistics/1394799/japan-frequency-of-personal-computer-use/
[6] - https://en.wikipedia.org/wiki/Earthquake_Early_Warning_(Japan)#Inaccuracies
[7] - https://uwiseismic.com/sp_faq/do-all-earthquakes-cause-tsunamis/
[8] - https://datareportal.com/reports/digital-2022-japan
[9] - https://globalquake.net/
[10] - https://jquake.net/en/
[11] - https://scratch.mit.edu/projects/636244032
[12] - https://nerv.app/en/
[13] - https://nerv.app/en/supporters.html
[14] - https://www.forbes.com/sites/johnkoetsier/2021/02/15/91-of-us-hate-being-forced-to-install-apps-to-do-business-costing-brands-billions/?sh=29821265da86
[15] - https://www.statista.com/statistics/218089/global-market-share-of-windows-7/
[16] - https://pubmed.ncbi.nlm.nih.gov/31765601/
[17] - https://www.j-shis.bosai.go.jp/intl/cjk/doc/meeting/1st/27_11_Japan.pdf (Page 10)
[18] - https://www.kyoshin.bosai.go.jp/kyoshin/docs/new_kyoshinmonitor.shtml#kmoni_useterms